

configure Agent Enhancement 与 OMX 的结构分析、优化与对照测试

configure 工程分析记录

2026 年 9 月 9 日

摘要

本文分析 `/Users/zhangxin/configure` 中面向 Codex、OpenCode 和 Claude 的 agent 增强部分，并与同一工作目录中引入的 OMX 项目 `refer/git-rep/oh-my-codex` 对照。工作对象的优化集中在统一启动器、跨平台运行时、会话恢复、分层上下文发现、生命周期事件、停止边界、并发锁、hook 适配和只读状态查询。结论按证据等级书写：configure 在低依赖、三类 CLI 统一入口、Unix/Windows 对齐、运行上限和可观测性方面形成了可复现优势；OMX 在多 worker 编排、tmux pane 权威、动态 scaling、worktree、角色路由、原子状态和大规模 hook 扩展方面仍明显更完整。因而本文不把轻量部署优势外推为整体平台超越，并给出已验证的优势范围与剩余差距。

目录

1	目的与前期准备	2
2	输入是什么	2
3	输出应该是什么	3
4	整体框架	3
5	关键细节	4
5.1	统一运行时状态机	4
5.2	上下文与 CLI 适配	4
5.3	hook 和质量边界	4
5.4	OMX 的关键扩展	4
6	任务如何正确执行	5
7	实际执行情况	5
8	实际输出情况	6

1 目的与前期准备	2
9 结果分析	6
9.1 能力矩阵	6
9.2 configure 的已证实优势	6
9.3 工作对象的劣势及其根因	6
10 综合分析	6
11 结尾总结	8
12 结尾评价	8
13 补充说明	8
14 参考源	9
15 附录与附件	10
15.1 A: 定量规模	10
15.2 B: 测试命令与退出码	10
15.3 C: 交付前检查记录	11
15.4 D: 后续可验收项	11

1 目的与前期准备

本次任务的对象是 configure 项目中的 agent enhancement，目标包括理解对照对象的优势、定位工作对象的劣势、实现针对性优化、运行对照测试，并生成可追溯报告。报告参考 `/Users/zhangxin/PyQCU/docs/report_multigrid_quda_pyqcu_20260906.tex` 的 A4 学术报告结构，采用 XeLaTeX 编译。分析主体属于代码/软件工作，因此使用 15 步代码分析框架；物理模型不属于本任务范围。

前置环境为 macOS、Bash、Git、Node/npm 和 XeLaTeX。configure 没有构建框架，Shell 校验以 `bash-n` 为主，agent 行为用 fake CLI 回归脚本覆盖。OMX 的 TypeScript 构建需要 Node 20 及以上；近期回归套件还需要 Rust/Cargo。确证 这些条件分别由 configure 根部 `AGENTS.md:36--45`、OMX 的 `package.json:1--70` 和实际命令结果支持。

2 输入是什么

输入分为四组。第一组是工作代码：`bin/agent.sh`、`bin/agent.bat`、共享运行时、状态查询脚本、hook 适配层及回归测试。第二组是运行参数：launcher 名称、模型、reasoning effort、OpenCode variant、输入文件、继续间隔、回合上限、运行时长上限、停止文件和 resume run id。第三组是持久状态：`data/runs/<run-id>/` 下的 manifest、state、事件、上下文、日志和输入清单。第四组是对照证据：OMX HEAD `0a43c09b1c4d7dbd03cfa7d9c4c42738d5d31584`、版本 0.21.4、源码与测试命令。

运行时输入的边界由 `bin/agent.sh:250--280`、`bin/agent.sh:535--570` 和 `bin/agent.sh:`

912--952 的参数解析共同定义；持久化字段由 `bin/agent-runtime.sh:62--104` 定义。**推断** 将所有 launcher 的输入收敛到同一个运行记录，是为了让外部查询不再依赖某一种 CLI 的原始日志格式。

3 输出应该是什么

预期输出有三层：

1. 用户层输出：TUI 或 headless agent 调用、紧凑的实时日志、错误状态和退出码。
2. 机器层输出：`manifest.env`、`state.env`、`events.jsonl`、`context.txt`、`agent.log` 和 `inputs.txt`；这些文件不执行、不进入工作树提交。
3. 交付层输出：测试结果、比较结论、`docs/analy_agent_configure_20260909.tex` 和同名 PDF，以及符合 tag 约定的 Git 版本记录。

`data/README.md:7--27` 明确了数据文件语义、默认路径和 Git 边界；`bin/agent-status.sh:9--17` 定义了只读状态查询接口。**确证** 本报告源文件是本次任务的分析产物，PDF 只有在后文编译和版式检查完成后才视为有效交付。

4 整体框架

`configure` 的 agent enhancement 由四层组成：入口层按调用名把 `cl/op/co` 分发到三条 CLI 路径；共享运行时层负责运行目录、锁、状态、事件和上下文；适配层负责每个 CLI 的 `headless/resume` 链；hook 层负责生命周期适配、路径保护和只读质量门禁。入口和适配的函数位置见 `bin/agent.sh:178--478`、`bin/agent.sh:480--834` 和 `bin/agent.sh:835--1229`。

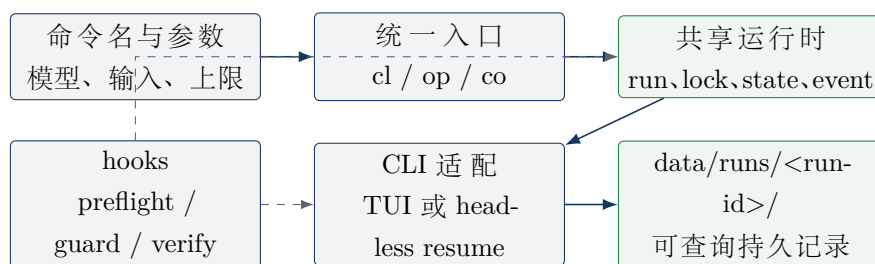


图 1: `configure agent enhancement` 的数据与控制关系。实线是主控制/持久化流，虚线是 hook 约束关系。

OMX 的层次更宽：CLI/skills 之上连接 session state、native hook、notifications 和 HUD；team 层再连接角色路由、任务 claim、worker mailbox、tmux pane、动态 scaling 与 worktree。其原生 hook 入口见 `codex-native-hook.ts:21888--22998`，team scaling 入口见 `scaling.ts:580--1921`；完整路径保留在参考源表中，因此两者不是同一规模的实现。

5 关键细节

5.1 统一运行时状态机

configure 先确定 workspace root, 再创建数据根和运行目录; 新运行写入 manifest/state 并加锁, resume 则先校验 schema、run id、agent、launcher 和 workspace, 再读取历史 session/thread、模型、reasoning 和 variant。状态转移包括 created、running、finished、stopped、failed 和 blocked, 回合开始、完成、失败及终态均追加 JSONL 事件。对应实现位于 bin/agent-runtime.sh:238--360、bin/agent-runtime.sh:373--496 和 bin/agent-runtime.sh:550--615。

锁使用 Bash 的 noclobber 创建; Unix 只在确认 owner PID 已退出时通过改名回收 stale lock, 并在释放时再次核对 owner 是否为当前 PID。该设计处理了两个极端情形: 活跃 owner 不可被误删, 旧 owner 不会永久阻塞恢复。实现和回归证据分别为 bin/agent-runtime.sh:126--167 与 bin/agent.test.sh:168--209。

5.2 上下文与 CLI 适配

上下文发现沿当前目录到 Git 根逐层收集 AGENTS.md、CODEX.md、CLAUDE.md 和 OPENCODE.md, 近处优先; 运行时只把路径和行数写入 context 清单, 正文仍由 agent 按需读取。实现见 bin/agent-runtime.sh:292--360, hook 的只读预检见 hooks/codex-preflight.sh:36--71。

Claude 通过 claude-p 和 --resume 绑定 session; OpenCode 通过持久日志提取 session id 并调用 run-s; Codex 通过 codexexec--json 提取 thread id 并调用 execresume。三条链的关键入口分别为 bin/agent.sh:345--463、bin/agent.sh:726--824 和 bin/agent.sh:1133--1209。模型显式覆盖优先, resume 默认复用原记录中的模型及相关推理设置。

5.3 hook 和质量边界

codex-hook.sh 把 session-start、summary、before-edit、after-edit、stop 和 notify 统一映射; --json 通过 codex-event.sh 产生包含 schema、event、workspace、run/session/thread、turn 和 status 的 JSONL envelope。路径 guard 使用结构化 JSON 解析, 拒绝仓库外、符号链接逃逸、日志和运行时数据路径。证据为 hooks/codex-hook.sh:14--103、hooks/codex-event.sh:14--62 和 hooks/codex-guard.sh:97--225。

5.4 OMX 的关键扩展

OMX 的 team 状态机由 orchestrator.ts:8--146 定义; 角色提示和任务路由由 role-router.ts:22--204 提供; 任务分配位于 allocation-policy.ts:110--212。worker runtime identity 的测试证明启动角色和模型 lane 会被保留, 见 worker-runtime-identity.test.ts.scaling.ts:109--276 和 scaling.ts:580--1430 进一步将锁、pane receipt、进程 PID、Team owner、任务和 worktree 纳入 scale-up 事务; worktree.ts:303--551 管理 worktree 规划、创建、回滚和删除。

OMX 的 native hook 还处理 session owner、子 agent、终态取消、prompt provenance、状态重建和 replay dedupe; 其关键入口可由 codex-native-hook.ts:772--893、codex-native-hook.ts:1662--2188 和 codex-native-hook.ts:4005--4108 看出。

`process-identity.ts:17--79`、`mode-binding-lease.ts:69--256` 和 `lifecycle-dedupe.ts:5--191` 分别覆盖进程身份、canonical session lease 和生命周期去重。

6 任务如何正确执行

本次正确执行路径如下。每一行都是一个可独立核验的动作，和实际脚本入口一一对应。

表 1: configure agent enhancement 的执行 workflow

步骤	动作与停止条件	实现证据
1	解析 launcher、模型和驱动参数；非法数字或时长立即失败。	<code>bin/agent.sh:250--280</code>
2	建立 workspace root、data 根和层级上下文清单。	<code>bin/agent-runtime.sh:238--360</code>
3	新建或恢复 run；恢复前身份校验通过才取得锁。	<code>bin/agent-runtime.sh:373--496</code>
4	绑定 CLI session/thread；写入 manifest/state 和 session-bound 事件。	<code>bin/agent-runtime.sh:531--539</code>
5	运行首回合；每回合写 turn-start，完成或失败后更新计数。	<code>bin/agent-runtime.sh:550--571</code>
6	检查 stop-file、max-turns、max-runtime 和连续失败阈值。	<code>bin/agent-runtime.sh:574--596</code>
7	到达终态后追加 session-end/session-stop，并仅释放当前 owner 锁。	<code>bin/agent-runtime.sh:598--615</code>
8	用 status、hook verify 和测试脚本检查持久输出及工作树。	<code>bin/agent-status.sh:99--176;</code> <code>hooks/codex-verify.sh:157--289</code>

推断 这条顺序把“调用 CLI”和“管理运行”解耦：CLI 失败不会抹掉失败原因，外部程序也能在不执行 state 文件的情况下读取状态。对长期运行任务，这比单纯依赖工作目录下的临时日志更容易恢复和审计。

7 实际执行情况

configure 实际执行了 Shell 语法、agent fake CLI 回归、hook 回归、工具检查、coverage 报告检查、推荐插件安装检查、tag 链检查、Windows 静态契约检查和 Git 空白检查。configure 的核心回归命令为 `bashbin/agent.test.sh` 和 `bashhooks/hooks.test.sh`，均返回 0。

OMX 先执行 `npmci` 和 `npmrunbuild`，均返回 0；`npmci` 报告 172 个包并提示 7 个 audit vulnerabilities (1 low、3 moderate、3 high)。随后运行三个聚焦套件：worker identity 与 cross-rebase 均通过；recent bug suite 在 `npmrunbuild:runtime` 阶段因当前环境没有 cargo 返回 127。精确命令 `npmruntest:explicit-terminal-contract` 没有定义，返回 1；实际存在的 `compiled` 命令运行后返回 1。

8 实际输出情况

configure 的核心回归输出为 15 项 agent PASS 和 11 项 hook PASS。覆盖内容包括 Codex once/resume、manifest run id 错配、stale/busy/non-owner lock、跨 agent/workspace 恢复拒绝、OpenCode 文件输入、Claude session 绑定、max-turns、max-runtime、blocked、stop-file、JSON status、无工作目录垃圾和 hook 路径安全。

OMX worker identity 输出为 4 tests、4 pass、0 fail，耗时约 3.09 s；cross-rebase 输出为 3 tests、3 pass、0 fail，耗时约 2.87 s。compiled explicit terminal suite 的前置文件通过，但 native hook 文件最终失败；可复查的失败包括 malformed exec follow-up queue、persisted subagent resume guidance、session scope drift、unmatched sidecar blocker、native stop replay dedupe 和 permission-style auto-nudge。该结果不是 configure 的失败，表示 OMX 当前对照环境中的该套件未达到全绿。

9 结果分析

9.1 能力矩阵

9.2 configure 的已证实优势

第一，统一 launcher 让三种 CLI 共用运行边界和持久布局，减少了用户为不同工具记忆不同恢复方式的成本。第二，数据写入默认落在 `${HOME}/configure/data`，工作目录不再产生新版 `.agent.*` 日志；这对配置仓库和多 workspace 场景尤其重要。第三，恢复身份校验覆盖 run id、agent、launcher 和 workspace，且模型参数有“原记录复用、显式参数优先”的确定规则。第四，Unix stale lock 的改名回收和 owner 二次校验直接覆盖了并发/异常退出的高风险边界。上述判断均由 `data/README.md:7--27`、`bin/agent-runtime.sh:126--193` 和 `bin/agent.test.sh:120--325` 直接支持。

9.3 工作对象的劣势及其根因

主要劣势是规模和语义深度，而不是当前核心回归缺陷。`bin/agent.sh` 已达 1237 行，Windows runtime 达 1650 行，Unix 和 Windows 仍存在部分参数/驱动逻辑重复；这会增加后续修复需要同步两侧的风险。其次，状态是单 run 级别的轻量文件状态，没有 OMX 那样的 owner identity、pane authority、worker claim、mailbox 和 worktree 事务；多 worker 协作若直接在其上叠加，容易把“进程活着”误当成“任务仍归属该 worker”。第三，hook 是显式调用适配层，当前不能宣称已经接入某个 Codex 官方统一生命周期发现 API；`hooks/README.md:1--3` 已明确这个边界。第四，Windows 只有静态契约验证，独占句柄、PowerShell 解析器和真实 CLI 链尚未实机验证。

10 综合分析

从项目思路看，configure 选择的是“把成熟 CLI 当执行引擎，在配置仓库增加一个薄的、可移植的运行控制面”。共享 runtime 用稳定的文件协议连接三种 agent，hook 用显式命令降低对外部 API 版本的依赖，数据目录隔离则把可观测性从工作区副作用中分离出来。这解释了它在 HPC 登录节点、没有 Node/Rust 的机器和简单无人值守任务上的实际优势。

表 2: 工作对象与 OMX 的 agent enhancement 对照

能力	configure 最终版	OMX HEAD	证据等级
统一入口	cl/op/co, 共享 Shell runtime; Windows batch 转 PowerShell	以 Codex 为执行核心的 npm CLI 与 plugin/setup 体系	确证 / bin/agent.sh: 178--1237; package.json: 1--70
会话恢复	run id + CLI session/thread, 身份校验和参数复用	canonical session、owner PID、子 agent/root 及更细的 session provenance	确证 两边均有, OMX 更完整
状态耐久性	原子替换 manifest/state, JSONL 事件, Unix stale lock	原子状态、lease、reconciliation、derived state 和更严格的终态契约	确证 / 推断
运行边界	max-turns、max-runtime、stop-file、blocked	explicit terminal contract、Stop hook 和 Autopilot/Ralph 状态机	确证 两边均有, 语义不同
上下文	从当前目录到仓库根分层发现四类说明文件	scoped AGENTS、role/tier/posture、prompt composition	确证 / 推断
hook	轻量显式 adapter、JSON envelope、路径 guard/verify	native lifecycle hook、通知、dedupe、MCP、插件扩展	确证
多 worker	未实现持久 mailbox、tmux authority、动态 scaling	team、任务 claim、worker mailbox、tmux pane、scale-up/down、worktree	确证
跨平台	Unix 与 Windows runtime 语义对齐; Windows 实机未验证	macOS/Linux 推荐 tmux, Windows 有受限路径	确证 / 未验证
依赖与部署	Bash + 原生 CLI, 低依赖, 可直接放入配置仓库	Node/npm, 部分 runtime/explore 需要 Rust/Cargo	确证
测试规模	15 agent + 11 hook 核心回归, 静态 Windows 契约	443 个 test files; 聚焦团队测试通过, native hook 聚焦套件未全绿	确证

OMX 选择的是“把 Codex 扩展成完整的工作流和多 agent 平台”。其角色/tier/posture prompt composition、native agent catalog、terminal contract、session reconciliation、worker mailbox、tmux authority、动态 scaling、worktree 和插件/hook 生态互相咬合；这些能力带来更高的协作上限，也带来 Node、tmux、状态协议和更大的测试/维护成本。README.md:36--42、README.md:121--167 以及上述 team/state 源码共同支持这一判断。

两者的核心差异可以写成状态空间的差别：configure 主要维护

$$S_{\text{cfg}} = (\text{run}, \text{cli session}, \text{limits}, \text{lock}, \text{events}), \quad (1)$$

而 OMX 还维护

$$S_{\text{omx}} = S_{\text{cfg}} + (\text{owner}, \text{pane}, \text{worker}, \text{task claim}, \text{mailbox}, \text{worktree}, \text{hook provenance}). \quad (2)$$

推断 第二个状态空间严格包含第一个的协作维度，因此 configure 在单进程/轻量部署上可以更简单、更快收敛，却不能据此声称在完整 team orchestration 上超过 OMX。

11 结尾总结

本次优化把 configure 从“按名称启动 CLI 的脚本集合”推进为“有统一运行协议的三 agent launcher”：运行创建、恢复、锁、上下文、回合上限、终态、事件和状态查询均有明确文件接口；Unix 与 Windows 有对应实现；hook 具备 JSON envelope 和 Git 路径质量边界。核心 configure 回归全部通过，OMX 的两个团队聚焦回归也通过。

最终判断是分场景的：在低依赖、跨 Codex/OpenCode/Claude、Unix/Windows 统一入口、显式停止和可观测性这组指标上，configure 已形成清晰优势；在多 worker 的身份权威、tmux 协作、动态伸缩、worktree 事务和 native hook 状态机上，OMX 仍然领先。报告不把未实现能力写成已完成能力。

12 结尾评价

13 补充说明

本报告有四类限制。其一，OMX 的 `test:recent-bug-regressions` 未能进入测试阶段，因为当前环境没有 `cargo`；不能据此判断 Rust runtime 本身通过或失败。其二，OMX 的 `compiled explicit terminal suite` 返回 1，报告只记录实际失败，不把未通过场景归因成单一原因。其三，Windows PowerShell parser、独占锁和真实 Codex/OpenCode/Claude CLI 没有在当前 macOS 上实机验证；Windows 结论限于静态契约。其四，已测的是无网络帮助入口启动时间，未测模型调用吞吐、首回合延迟或恢复延迟；因此启动数据只说明本机轻量入口开销，不能写成统一性能定律。

此外，configure 的 hook 不自动修改 Codex 配置或 `core.hooksPath`；启用方式仍由调用方 runner 决定，见 `hooks/README.md:39--51`。运行时 JSONL 的安全性依赖当前 Shell escape 实现，后续若允许任意非 ASCII 或更复杂的控制字符，应增加 jq/Python 解析回读测试。

表 3: 优化评价与优先级

维度	当前评价	后续优先级
可靠性	恢复身份、锁 owner、运行上限和终态已有回归证据。	继续增加进程身份 token 和状态并发故障注入。
可维护性	共享 runtime 已抽出； Unix/Windows 仍有较大重复。	把参数 schema 和事件字段抽为跨平台契约，减少双写。
可观测性	manifest/state/events/status 构成完整轻量闭环。	增加事件 schema 版本迁移和 redaction 检查。
部署性	Bash + 原生 CLI，适合没有 Node/Rust 的工作站/HPC。	在 Windows runner 和真实 CLI 上完成实机矩阵。
协作上限	单 run 边界可靠，多 worker 编排能力不足。	若需求扩大，再引入 mailbox/worker claim/worktree，而不是把状态字段无限堆到单 run。

14 参考源

来源	用途与关键行号
AGENTS.md:1--45	仓库边界、目录职责、测试惯例和 agent 入口。
bin/AGENTS.md:29--47	launcher、数据目录、参数和跨平台约定。
bin/agent.sh:178--1237	Claude/OpenCode/Codex 分发、驱动和参数解析。
bin/agent-runtime.sh:62--615	manifest、state、lock、context、事件、恢复和终态。
bin/agent-status.sh:9--176	只读状态查询和 JSON 输出。
hooks/codex-hook.sh:14--103	hook 事件映射和 JSON 模式。
hooks/codex-event.sh:14--62	统一 JSONL event envelope。
hooks/codex-guard.sh:97--225	仓库边界、符号链接和运行数据保护。
hooks/codex-preflight.sh:36--71	分层上下文只读预检。
hooks/codex-verify.sh:157--289	Git、Shell、日志和数据路径验证。
bin/agent.test.sh:120--325	configure agent fake CLI 回归。
hooks/hooks.test.sh:1--227	hook 质量门禁回归。
data/README.md:7--27	运行时数据协议和隔离规则。
refer/git-rep/oh-my-codex/package.json:1--70	OMX 版本、构建、测试和依赖入口。
refer/git-rep/oh-my-codex/src/team/orchestrator.ts:8--146	team 状态机与 phase agent。
refer/git-rep/oh-my-codex/src/team/scaling.ts:109--1430	claim lock、pane authority、scale-up 事务。

来源	用途与关键行号
refer/git-rep/oh-my-codex/src/team/worktree.ts:303--551	worktree 规划、创建、回滚和删除。
refer/git-rep/oh-my-codex/src/state/process-identity.ts:17--79	进程 owner token 和启动身份。
refer/git-rep/oh-my-codex/src/state/mode-binding-lease.ts:69--256	canonical mode binding lease。
refer/git-rep/oh-my-codex/src/notifications/lifecycle-dedupe.ts:5--191	lifecycle 通知去重。
refer/git-rep/oh-my-codex/src/scripts/codex-native-hook.ts:772--22998	native hook 映射、session、终态和分发。
refer/git-rep/oh-my-codex/src/team/__tests__/worker-runtime-identity.test.ts	worker runtime identity 聚焦测试。
refer/git-rep/oh-my-codex/src/team/__tests__/cross-rebase-smoke.test.ts	cross-rebase 聚焦测试。
hooks/README.md:1--51	hook API 边界和显式启用方式。

15 附录与附件

15.1 A：定量规模

截至对照 HEAD，configure 全仓库 Git 跟踪文件数为 3271；本次新增/修改的 agent 相关文件共 17 个，按交付集合统计 5139 行，其中 agent.sh 1237 行、Unix runtime 616 行、PowerShell runtime 1650 行。OMX 的 src/ 共 853 个文件、446080 行，其中 443 个测试文件；非测试源码文件 410 个。OMX team/state/notify/CLI 选定核心集合约 44863 行。此处行数是工程规模指标，不是质量分数。

15.2 B：测试命令与退出码

命令	退出码	实际结果
bash-nbin/*.shhooks/*.shtools/*.shplugins/*.shskills/*/*.sh	0	通过
bashbin/agent.test.sh	0	15 PASS
bashhooks/hooks.test.sh	0	11 PASS
bashtools/configure-check.test.sh	0	通过；原有 plugins warning，无 error

命令	退出码	实际结果
bashtools/coverage-report.test.sh	0	通过
bashplugins/install-recommended.test.sh	0	通过
bashskills/tag/tag-chain.test.sh	0	通过
bashbin/agent-windows.test.sh	0	静态契约通过；实机跳过
gitdiff--check	0	通过
npmci (OMX)	0	172 packages; 7 vulnerabilities
npmrunbuild (OMX)	0	TypeScript 构建通过
npmruntest:team:worker-runtime-identity (OMX)	0	4/4 通过
npmruntest:team:cross-rebase-smoke (OMX)	0	3/3 通过
npmruntest:recent-bug-regressions (OMX)	127	缺少 cargo, 停在 build:runtime
npmruntest:explicit-terminal-contract (OMX)	1	script 不存在
npmruntest:explicit-terminal-contract:compiled (OMX)	1	native hook 聚焦套件未全绿

无网络帮助入口启动的 3 次测量为：configure 使用 `bashbin/co--help`，耗时 **0.08–0.09 s**，退出码均为 0；OMX 使用 `nodedist/cli/omx.js--help`，耗时 **0.29–0.33 s**，退出码均为 0。该测量包含独立进程启动和参数帮助路径，不包含模型请求、网络、首回合、session 恢复或 team 建立；它支持“低依赖入口更轻”的局部判断，不能替代完整性能基准。

15.3 C：交付前检查记录

报告编译应使用以下闭环：在临时目录以 XeLaTeX 编译两遍，确认 PDF 非空；用可用的 PDF 工具检查页数和文本；若工具链不完整，明确写入未验证状态。`xelatex` 和 Ghostscript 在当前环境可用，`pdfinfo`、`pdftoppm`、`mutool` 和 `qpdf` 未发现，因此本次页数由 XeLaTeX/PDF 文件和文本提取检查确认，逐页图像渲染指标若无法取得则标为未验证，不冒充人工视觉通过。

15.4 D：后续可验收项

1. 在 Windows 或安装 PowerShell 的 CI 上运行 `bin/agent-windows.test.sh` 的实机分支，覆盖文件句柄锁、resume 和三类真实 CLI。
2. 将 Unix/PowerShell 的参数和 event schema 生成自同一份契约，消除两侧重复解析。
3. 若需要多 worker，再以独立的 worker identity、task claim、mailbox 和 worktree 事务为一组引入，并为每个状态转移增加故障注入测试。
4. 在同一机器、同一 fake CLI 和相同输入下重复测量启动、首回合、恢复和状态查询延迟，再决定是否需性能优化。